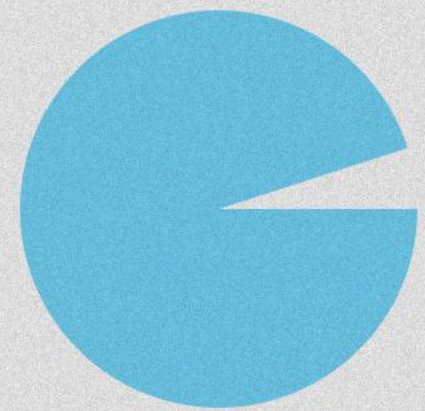
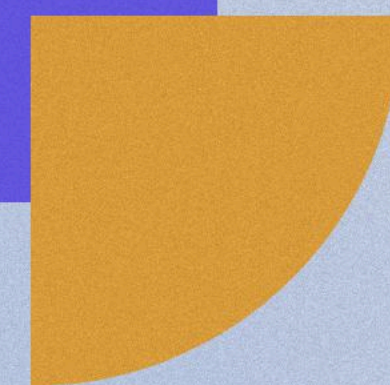
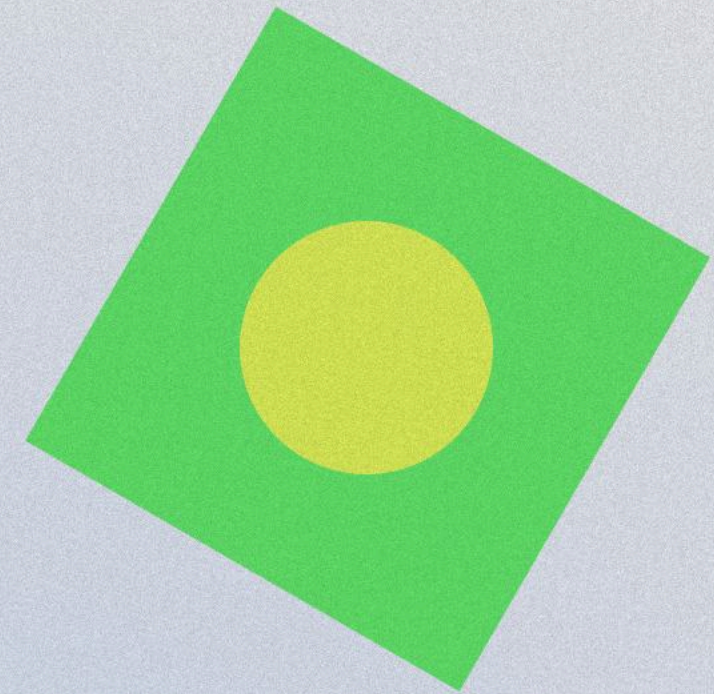




ВСПОМИНАЕМ

Structured  
Query  
Language



# НЕСКОЛЬКО ОБЯЗАТЕЛЬНЫХ ТЕРМИНОВ...

1

**База данных (Data Base):** Упорядоченный набор структурированной информации.  
• Пример: Телефонная книга, каталог в библиотеке.

2

**СУБД (Database Management System):** Инструмент для управления.  
• Примеры: PostgreSQL, MySQL, SQLite (легкие), Oracle (тяжелые).

3

**Зачем нужна СУБД?**

- Хранение больших объемов.
- Быстрый поиск (индексы).
- Безопасность (логины/пароли).
- Одновременная работа сотен пользователей (транзакции).

# Основные понятия

Таблица = Excel-лист.

Строка (Row/Record) = Одна конкретная запись (например, один клиент).

Колонка (Column/Field) = Атрибут этой записи (например, Имя, Дата рождения).

Первичный ключ (Primary Key / PK): Уникальный идентификатор каждой строки.



| Tables (10) |                |
|-------------|----------------|
| >           | classrooms     |
| >           | events         |
| >           | groups         |
| >           | homework       |
| >           | news           |
| >           | schedule_items |
| >           | subjects       |
| >           | teachers       |
| >           | user_groups    |
| >           | users          |



|    | id<br>[PK] uuid | email<br>character varying (255) | password_hash<br>character varying (255) | name<br>character varying (255) | role<br>enum_users_role | avatar_url<br>character varying (255) | createdAt<br>timestamp with time zone | updatedAt<br>timestamp with time zone |
|----|-----------------|----------------------------------|------------------------------------------|---------------------------------|-------------------------|---------------------------------------|---------------------------------------|---------------------------------------|
| 0+ | 1               | example@mail.ru                  | asiufuUI@UlulAFulaul...                  | Vladislav                       | user                    | [null]                                | [null]                                | [null]                                |



# ТИПЫ ДАННЫХ

BOOLEAN

1

логические значения `true` (истина) и `false` (ложь).

VARCHAR(n)

2

Строки переменной длины (Имена, адреса)

TEXT

3

Длинный текст (посты, комментарии)

DATE / DATETIME

4

Только дата / Дата + время

DECIMAL

5

Деньги/цены (точное число)

INTEGER

6

Целые числа (счетчики, возраст)

# Этапы создания базы данных

---

Этап 1. Сбор требований и определение сущностей

1

Этап 2. Типы связей

2

Этап 3. Нормализация

3

# Этап 1. Сбор требований и определение сущностей

---

Как определить сущность?

Сущность – это самостоятельный объект, о котором мы храним информацию.

Необходимо задать вопрос: "Кто" или "Что" участвует в процессе?

Пример для интернет-магазина:

1. Кто покупает? → Покупатель (сущность).
2. Что покупает? → Товар (сущность).
3. Где хранить? → Склад (сущность).
4. Что делает? → Заказ (сущность – событие).

Для каждой сущности создаем отдельную таблицу.

## Этап 2. Типы связей

---

### Один к одному (1:1)

- Пример: Покупатель и Паспортные данные.
- Логика: Один покупатель = одни паспортные данные.



### Один ко многим (1:M)

- Пример: Один покупатель делает Много заказов.
- Логика: В таблице "Заказы" мы храним внешний ключ (FK) – `customer_id`, который ссылается на `id` из таблицы "Покупатели".



### Многие ко многим (M:N)

- Пример: Много товаров входят в Много заказов (кроссовки и футболка в одном заказе).
- Решение: Нельзя хранить список товаров прямо в заказе. Создаем промежуточную таблицу (Связующую): `Заказы_Товары`, где каждая строка – это пара (`ID_Заказа`, `ID_Товара`).



## Этап 3. Нормализация

Избавляемся от дублирования данных.

| Покупатель | Товар    | Цена |
|------------|----------|------|
| Иван       | Носки    | 100  |
| Иван       | Футболка | 500  |

Если Иван купил 100 товаров, мы 100 раз напишем его имя. Если он сменит имя, придется править 100 записей.

Хорошо (Две таблицы):

- Таблица Покупатели: (ID=1, Имя="Иван")
- Таблица Покупки: (ID\_товара, ID\_покупателя=1)
- Результат: Имя хранится в единственном экземпляре.

## Язык запросов SQL

```
CREATE TABLE customers (  
    id INT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(255) UNIQUE,  
    created_at DATE DEFAULT NOW()  
);
```



C

INSERT (Добавить) – буква "C"

```
INSERT INTO customers (name, email) VALUES ('Алексей', 'alex@mail.ru');
```

R

SELECT (Прочитать) – буква "R" (Самый частый)

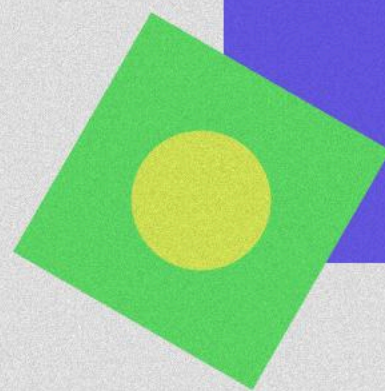
```
SELECT * FROM customers;
```

```
SELECT name, email FROM customers WHERE id = 1;
```

U

UPDATE (Обновить) – буква "U"

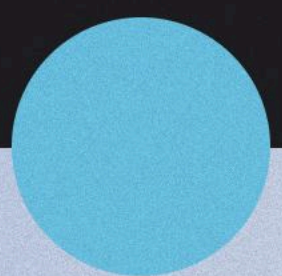
```
UPDATE customers SET email = 'new_email@mail.ru' WHERE name = 'Алексей';
```



D

DELETE (Удалить) – буква "D"

```
DELETE FROM customers WHERE id = 5;
```



# Сложный запрос – JOIN (Объединение таблиц)

```
SELECT
    orders.id AS "Номер заказа",
    customers.name AS "Имя клиента",
    orders.product AS "Товар"
FROM orders
JOIN customers ON orders.customer_id = customers.id;
```

- JOIN – берем строку из orders.
- ON – говорим: найди в таблице customers строку, где customers.id равен orders.customer\_id.
- Подставь данные из этой найденной строки в результат.